MGM University, Chh. Sambhajinagar

Jawaharlal Nehru Engineering College

Department of Computer Science & Engineering

LAB MANUAL

Program (UG/PG) :	UG
Year :	Third Year
Semester :	V
Course Code :	CSE21PCP302
Course Title :	Operating System Lab
Prepared By :	Mrs. Poonam M.Ramgirwar, Mr.Pramod Bhalerao

Department of Computer Science & Engineering

2025-26

FOREWORD

It gives me immense pleasure to present this Laboratory Manual for Third Year Engineering students for the subject of Introduction to Operating System Lab with guidelines of NEP 2020.

This manual is designed to provide clarity and guidance on the key concepts, practical implementations, and problem-solving strategies related to the subject. Students are encouraged to explore beyond the prescribed experiments, reflect on each exercise, and strengthen their understanding through hands-on experience.

At MGM University, we continuously strive to enhance the learning experience of our students. The institution is proud to be ISO 9001:2015 and 14001:2015 certified, and we believe in leveraging systematic academic practices for overall student development. We emphasizes that students should actively utilize this lab manual not just as a procedural document, but as a stepping stone to sharpen their technical skills. The knowledge and exposure gained through these experiments will contribute significantly to building a strong foundation for industry-readiness and innovation.

Faculty members are encouraged to facilitate conceptual understanding during lab sessions, ensuring that students develop a deep insight into the subject matter. Once concepts are clear, students will engage with enthusiasm, easing the teaching process and promoting self-directed learning.

Let this manual be a guide for skill enhancement and a tool for excellence in your academic journey.

Dr. Vijaya B Musande
Principal

LABORATORY MANUAL CONTENTS

This manual is intended for the Third year students of Computer Science and Engineering in the subject of Operating System. This manual typically contains practical/Lab Sessions related Operating System covering various aspects related the subject to enhanced understanding.

Students are advised to thoroughly go through this manual rather than only topics mentioned in the syllabus as practical aspects are the key to understanding and conceptual visualization of theoretical aspects covered in the books.

Good Luck for your Enjoyable Laboratory Sessions.

Mrs.Poonam M.Ramgirwar
Mr.Pramod Bhalerao
Subject Teacher

Dr. Deepa Deshpande
HOD

LIST OF EXPERIMENTS

Course Code: CSE21PCP302
Course Title: Operating ystem

S.No	Name of the Experiment
1.	a) To Install Ubuntu Linux and LINUX Commands (File Handling utilities, Text processing utilities, Network utilities, Disk utilities, Backup utilities and Filters).
	b) Write a Shell Script that accepts a file name, starting and ending line numbers as arguments and displays all lines between the given line numbers.
	c) Write a shell script that deletes all lines containing the specified word in one or more files supplied as arguments to it.
2.	a) Write a shell script that receives any number of file names as arguments checks if every argument supplied is a file or directory and reports accordingly. Whenever the arguments a file it reports no of lines present in it.
	b) Write a shell script that accepts a list of file names as its arguments, counts and reports the occurrence of each word that is present in the first argument file on other Argument files.
3.	a) Write a shell script to list all of the directory files in a directory
	b) Write a shell script to find factorial of a given number.
4.	a) write an awk script to count number of lines in a file that does not contain vowels.
	b) write an awk script to find the no of characters ,words and lines in a file
5.	Implement in c language the following Unix commands using system calls a) cat b) ls c) mv
6.	Write a C program that takes one or more file/directory names as command line input and reports following information a) File Type b) Number Of Links c) Time of last Access d) Read ,write and execute permissions
7.	Write a C program to list every file in directory, its inode number and file name.
8	a)Write a C program to create child process and allow parent process to display“parent”and the child to display “child” on the screen
	b) Write a C program to create zombie process
	c) Write a C program to illustrate how an orphan process is created.

9	a) Write a C program that illustrate communication between two unrelated process usingnamed pipes
	b) Write a C program that receives a message from message queue and display them
10	a) Write a C program to allow cooperating process to lock a resource for exclusive use(using semaphore)
	b) Write a C program that illustrate the suspending and resuming process using signal
	c) Write a C program that implements producer-Consumer system with two process using semaphore
11	Write client server programs using c for interaction between server and client process using Unix Domain sockets
12	Write a C program that illustrates two processes communicating using Shared memory

DOs and DON'Ts in Laboratory:

1. Make entry in the Log Book as soon as you enter the Laboratory.
2. All the students should sit according to their roll numbers starting from their left to right.
3. All the students are supposed to enter the terminal number in the log book.
4. Do not change the terminal on which you are working.
5. All the students are expected to get at least the algorithm of the program/concept to be implemented.
6. Strictly observe the instructions given by the teacher/Lab Instructor.
7. Do not disturb machine Hardware / Software Setup.
8. Uniform & I-Card are compulsory.
9. Do not use mobile during lab session.

Instruction for Laboratory Teachers:

1. Submission related to whatever lab work has been completed should be done during the next lab session along with signing the index.
2. The promptness of submission should be encouraged by way of marking and evaluation patterns that will benefit the sincere students.
3. Continuous assessment in the prescribed format must be followed.

Experiment No: 1

Aim:

a) **To Install Ubuntu Linux and LINUX Commands (File Handling utilities, Text processing utilities, Network utilities, Disk utilities, Backup utilities and Filters).**

Theory:

Ubuntu is a popular, free, and open-source operating system based on the Linux kernel. It is widely used for desktops, servers, and cloud computing. Ubuntu provides a stable and user-friendly environment for both beginners and advanced users, supporting a wide range of software and hardware.

Why Use Linux?

- **Open Source:** Anyone can view, modify, and distribute the source code.
- **Security:** Linux is known for strong security features and less vulnerability to viruses.
- **Stability:** Ideal for servers and critical systems due to its reliability.
- **Customization:** Highly customizable with various desktop environments and tools.
- **Community Support:** Large user community and extensive documentation.

Ubuntu Installation Overview

Installing Ubuntu involves preparing your system to boot from installation media (like a USB drive), selecting installation preferences, and copying system files onto your hard drive. The installation process sets up partitions, installs a bootloader (GRUB), and configures user accounts.

Basic Linux Command Categories

- **File Handling Utilities:** Manage files and directories (e.g., ls, cp, mv).
- **Text Processing Utilities:** Work with text files and data streams (e.g., grep, awk, sed).
- **Network Utilities:** Diagnose and manage network connections (e.g., ping, ssh).
- **Disk Utilities:** View and manage disk usage and partitions (e.g., df, mount).
- **Backup Utilities:** Create backups and clone data (e.g., tar, rsync).
- **Filters:** Process streams of text to manipulate or extract data (e.g., sort, uniq).

Why Learn Linux Commands?

The command line interface (CLI) allows precise control of the system and automation of tasks. Mastery of Linux commands enhances productivity, especially in server management, software development, and system administration.

Requirements for Installing Ubuntu Desktop

- A laptop or PC with at least **25GB** of storage space.
- System running on **Windows/MacOS**.
- A USB flash drive of **12 GB** or above.
- RAM **2GB.**
- **64-bit** processor of minimum **1GHz** clock speed.

Install Ubuntu on Windows and MacOS

Because of the attractive features or for a special purpose, you may need to install Ubuntu. The installation steps of Ubuntu are almost the same for every OS. Here, I'm demonstrating each of the steps to install Ubuntu on **Windows** and **MacOS:**

1.Back-up Your Data

While replacing your current OS with Ubuntu, make sure you have a **backup of your important files** to prevent data loss before proceeding to the installation steps**.**

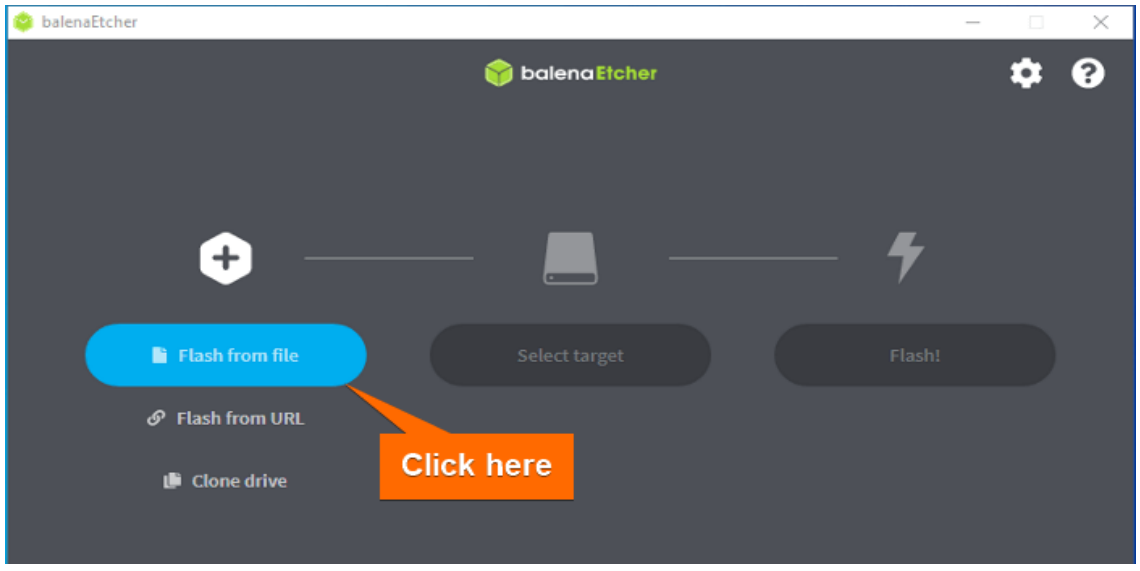
2. Download an Ubuntu ISO File

The first and easiest step of any OS installation is to download the ISO file. You can **download Ubuntu** from their official site. I've downloaded **Ubuntu 22.04.1 LTS**. Don't worry if you are downloading any other version. The later installation steps are the same for any version of Ubuntu**.**

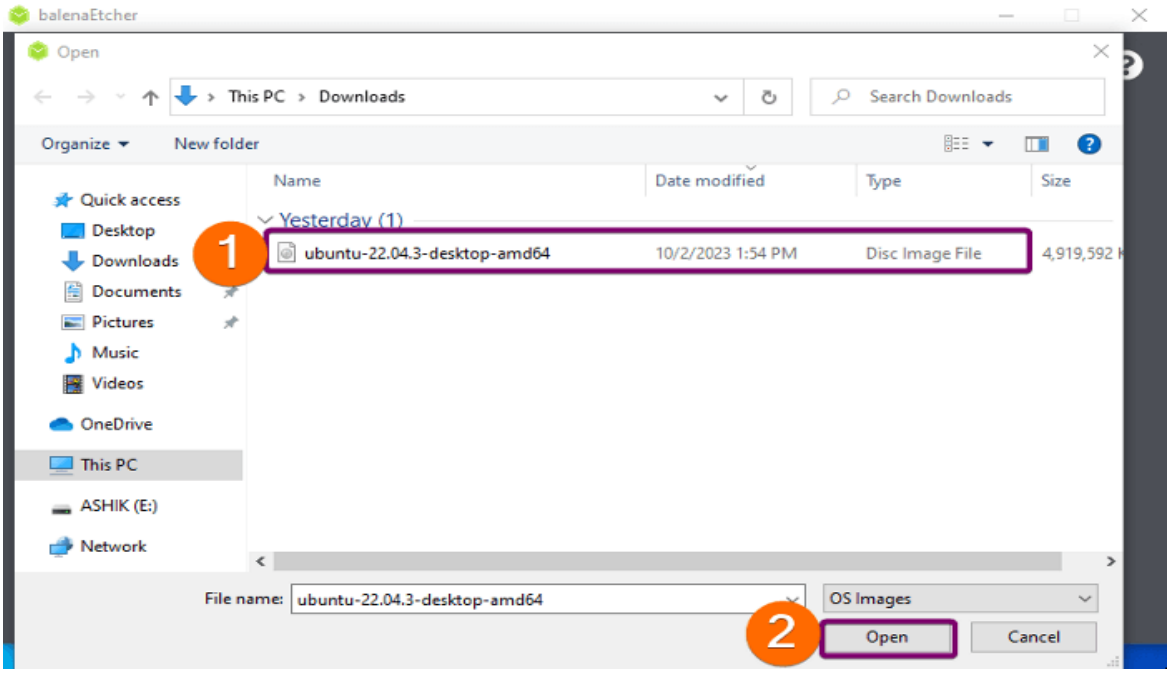
3. Create a Bootable USB Flash Drive

To create a bootable USB flash drive with the ISO file, I'll use **Etcher** which is a free and open-source application. First, **Install Etcher on Windows** or **MacOS**. Then, follow these steps to learn **how to use Ethcer** to create a bootable drive for installing Ubuntu:

1.Open Etcher and select "Flash from File

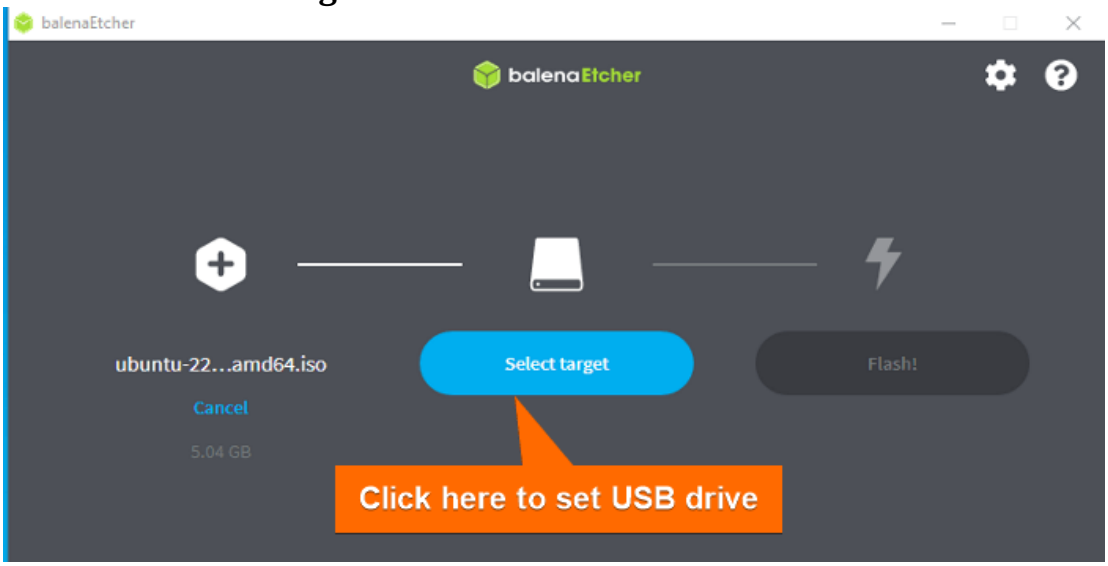


2.Select the **ISO file** from the location you've downloaded.

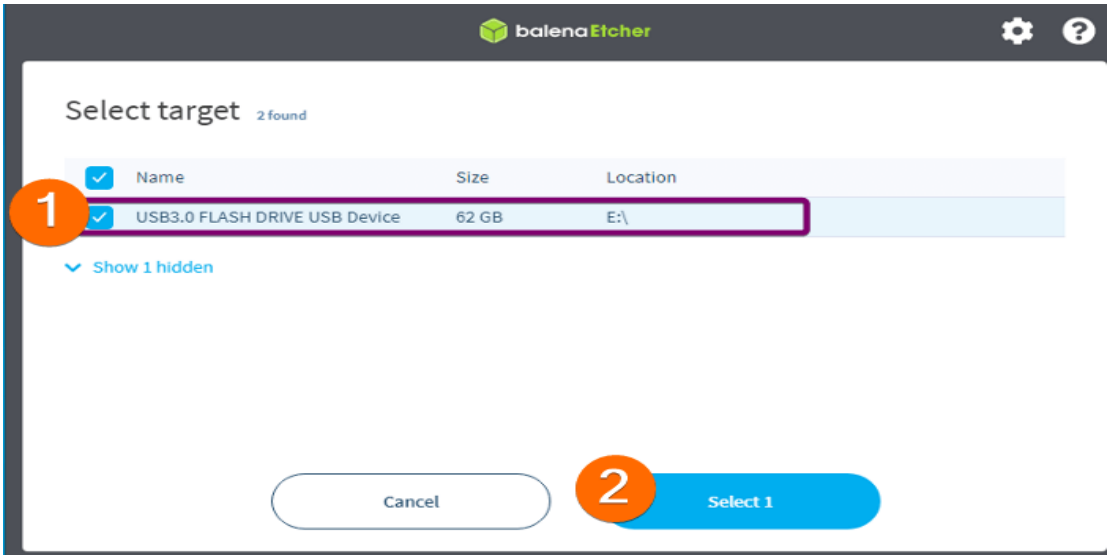


3.Plug in your USB pen drive to the PC.

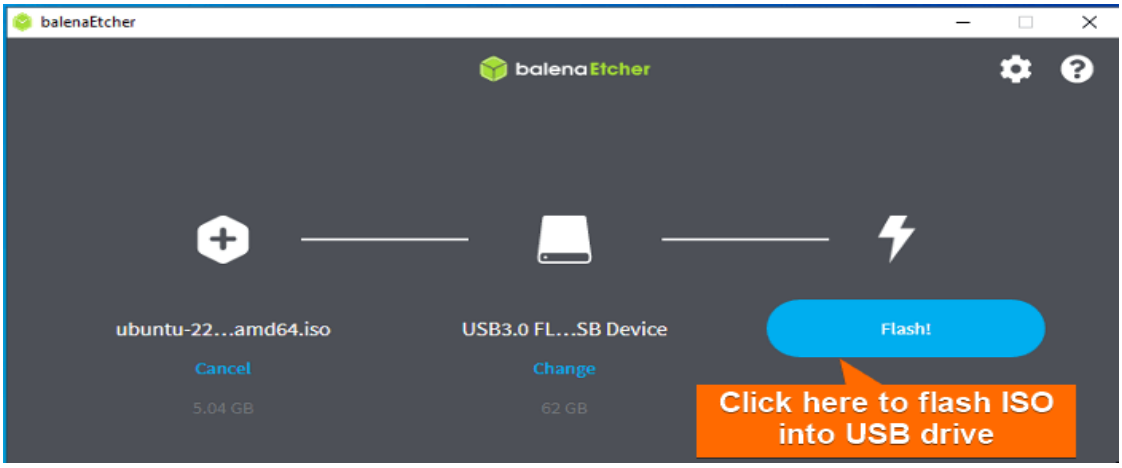
4.Click on **Select target.**



5.Select your USB drive.



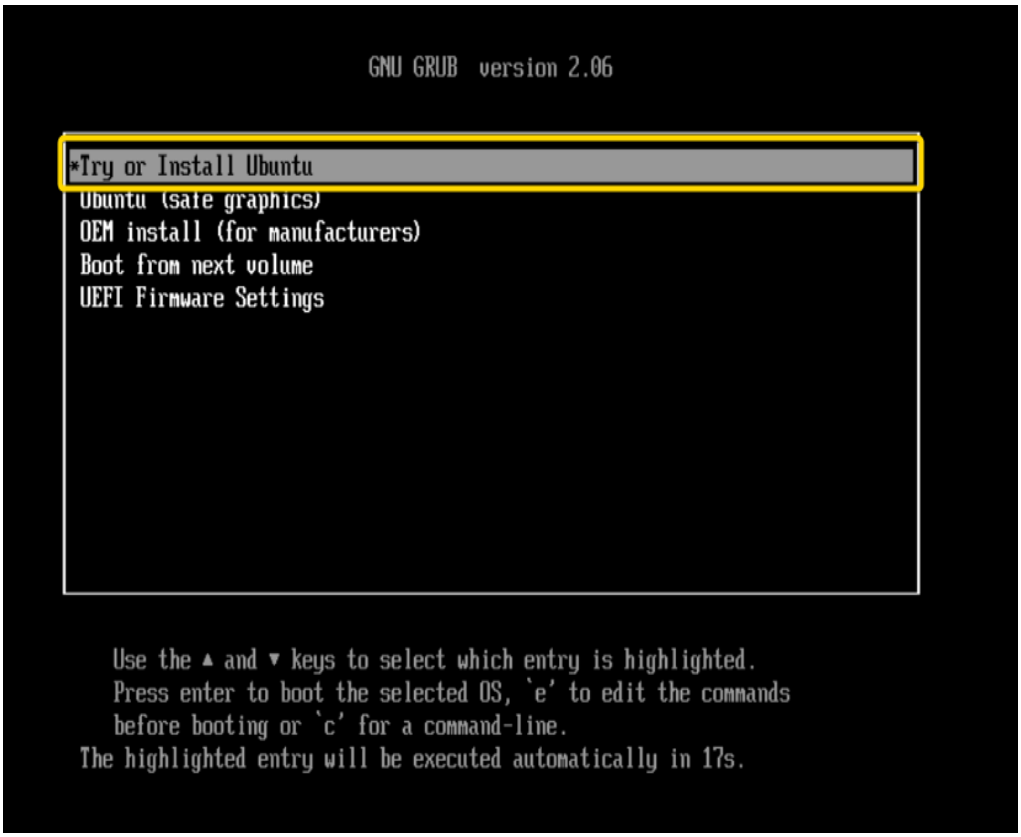
6.Click on **Flash** to start flashing the ISO file into the USB drive.



After that, Etcher will flash the ISO file into your USB drive and make it bootable.

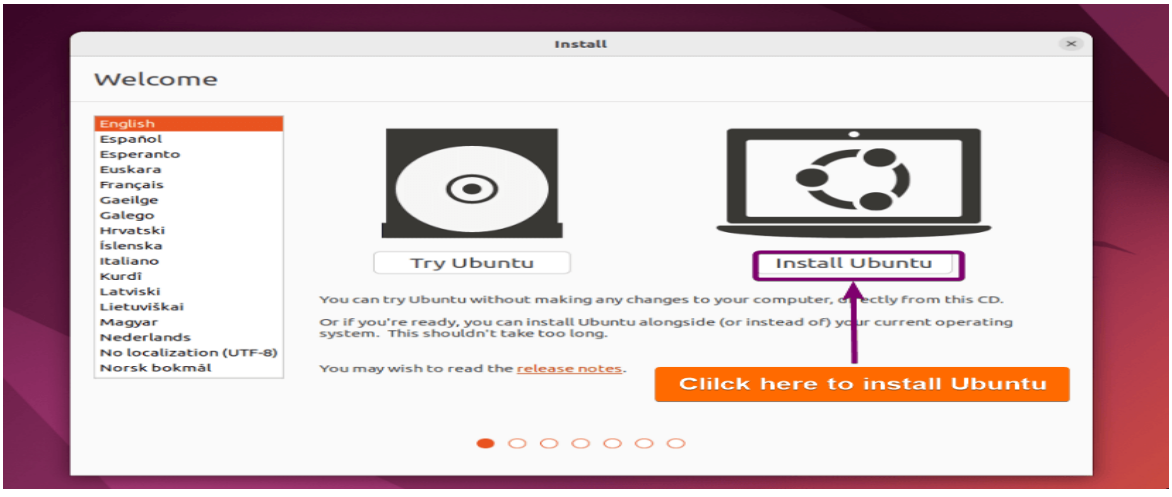
4. Boot From USB Flash Drive

After creating the bootable USB flash drive, plug it into the PC where you want to install the Linux distro. Then power on the PC. It will automatically launch the setup window like this.



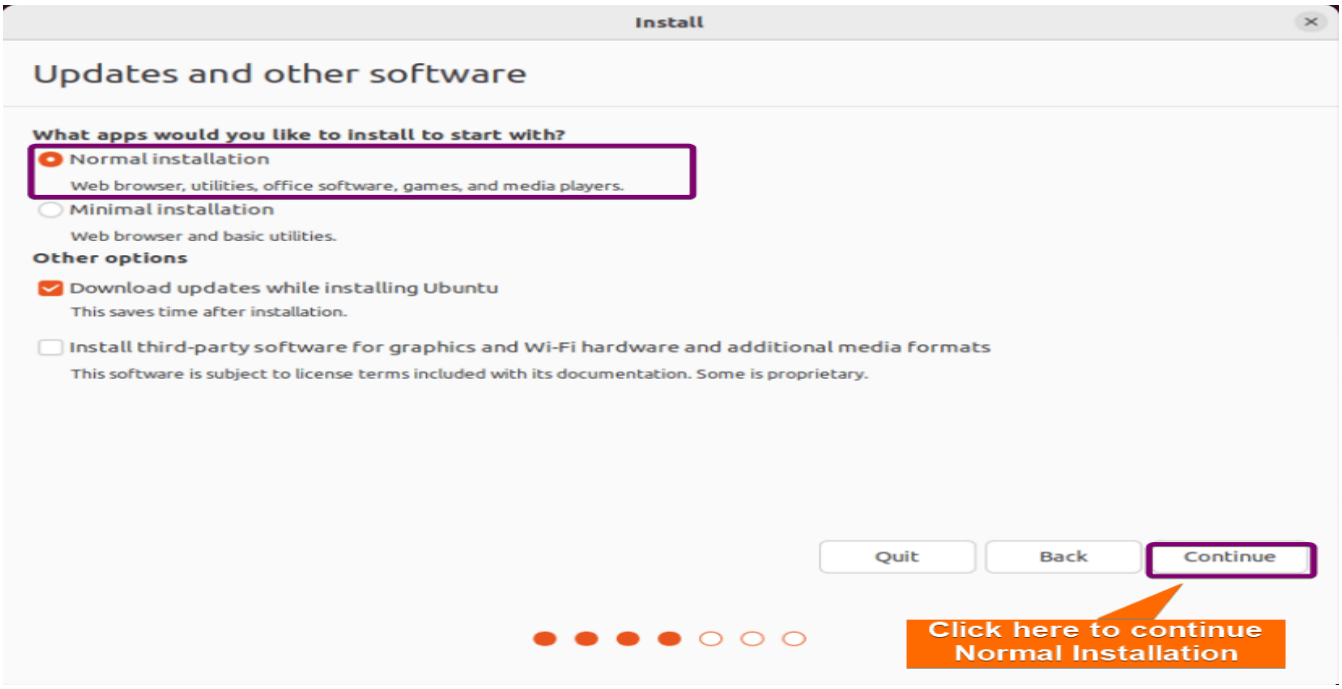
5. Start Installation

Now, a prompt will appear that allows you to choose between “Try Ubuntu” and “Install Ubuntu”. Click on **Install Ubuntu.**



6. Installation Setup

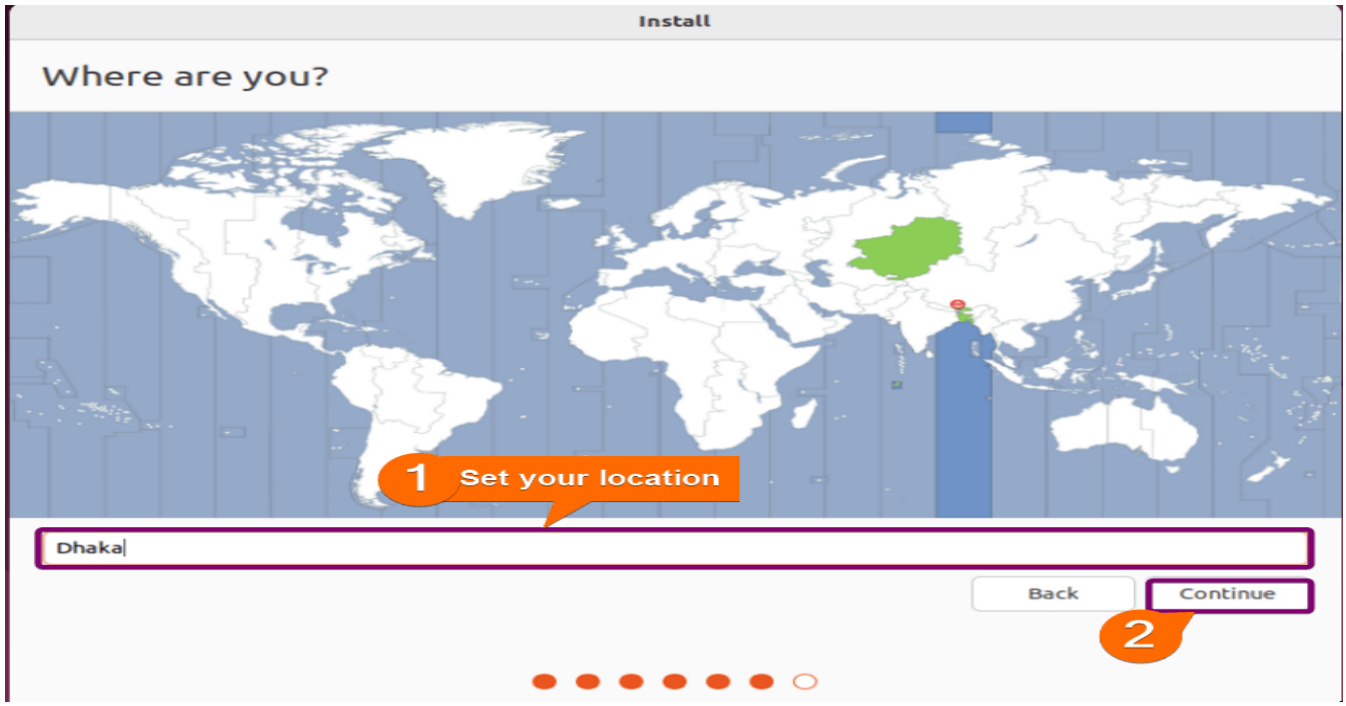
Now, a prompt will appear allowing you to choose between the Normal installation and Minimal installation options. The **normal installation** offers a **full-featured** desktop environment with **pre-installed software** and a familiar user interface, choose the normal installation. On the other side, **Minimal installation** only installs a web browser and some very basic utilities. If you are a **beginner**, I recommend choosing **Normal installation**.



9. Set Your Location

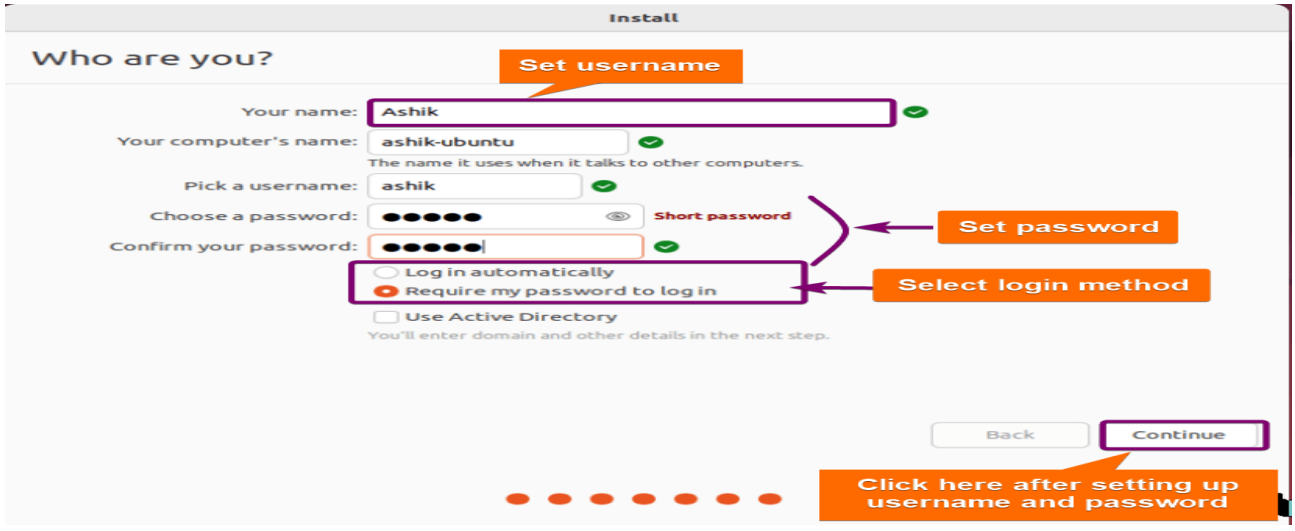
At this stage, you have to set your location. This information will be detected automatically if you are connected to the internet. However, you can **select your location and time zone** from the drop-down menu on the map screen. Click

on **Continue**.



10. Create Login Credentials

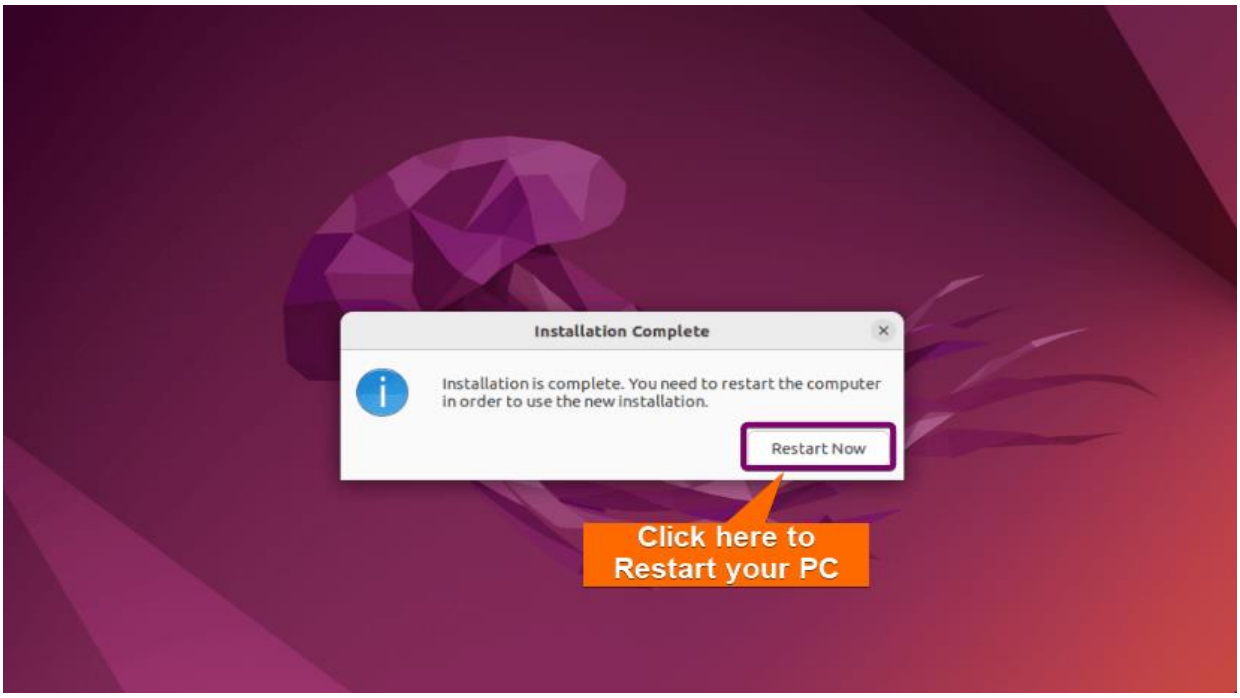
On this screen, you will be asked to enter your name and the name of your computer as it will appear on the network. Then, create a username and a strong password. Finally, select the login method. "Log in automatically" will not require a password when you start your PC. However, "Require my password to log in" will ask for the password you set **every time** you power on your PC.



11. Restart the PC to Complete the Installation

Now, it will take a moment and complete all the necessary setup automatically. At last, a restart prompt will appear. Click on **Restart**

Now.



After restarting, you can use Ubuntu with all

Conclusion

During the Ubuntu installation process, you can choose various options and settings, such as partitioning your disk, selecting software packages, configuring the user account, and more. However the exact steps for installing Ubuntu may vary depending on your hardware, but the general process is the same for all platforms.

1.2 Aim: Write a Shell Script that accepts a file name, starting and ending line numbers as arguments and displays all lines between the given line numbers.

Theory:

Shell Script Theory

What is a Shell Script?

- A **shell script** is a plain text file containing a sequence of commands for a Unix/Linux shell to execute.
- It automates repetitive tasks, simplifies complex commands, and can take inputs (arguments) to perform flexible operations.

How does it work?

- When you run a shell script, the shell (like bash, sh, or zsh) reads the file line by line and executes the commands in order.
- The first line of a script usually starts with a **shebang** (#!/bin/bash), which tells the system which interpreter to use to run the script.

Code:

Shell Script: display_lines.sh

```
bash
CopyEdit
#!/bin/bash

# Check if exactly 3 arguments are passed
if [ "$#" -ne 3 ]; then
  echo "Usage: $0 filename start_line end_line"
  exit 1
fi

filename=$1
start_line=$2
end_line=$3

# Check if file exists
if [ ! -f "$filename" ]; then
  echo "Error: File '$filename' not found."
  exit 1
fi

# Check if start_line and end_line are integers
if ! [[ "$start_line" =~ ^[0-9]+$ ]] || ! [[ "$end_line" =~ ^[0-9]+$ ]]; then
  echo "Error: Start and end lines must be positive integers."
  exit 1
fi

# Check if start_line is less than or equal to end_line
if [ "$start_line" -gt "$end_line" ]; then
  echo "Error: Start line number must be less than or equal to end line number."
  exit 1
```

```
fi

# Display the lines from start_line to end_line
sed -n "${start_line},${end_line}p" "$filename"
```

Output:

Aim: 1.3 Write a shell script that deletes all lines containing the specified word in one or more files supplied as arguments to it.

Theory

- The script accepts a **word** and one or more **file names** as command-line arguments.
- It processes each file and deletes lines containing the specified word.
- We use the sed command for in-place editing:sed -i '/word/d' filename deletes all lines containing "word" in filename.
- The script checks if arguments are provided and if the files exist.
- Using sed -i modifies files directly, so it's destructive. Always backup important files before running such scripts.

Program:

```
#!/bin/bash

# Check if at least two arguments are passed (word + at least one file)
if [ "$#" -lt 2 ]; then
  echo "Usage: $0 word file1 [file2 ...]"
  exit 1
fi

word=$1
shift  # Shift arguments so now $@ contains only file names
# Loop over all files
for file in "$@"; do
  # Check if file exists and is writable
  if [ ! -f "$file" ]; then
    echo "Error: File '$file' not found."
    continue
  elif [ ! -w "$file" ]; then
    echo "Error: File '$file' is not writable."
    continue
  fi

  # Delete lines containing the word using sed (in-place)

  sed -i "/$word/d" "$file"

  echo "Processed file: $file"
```

```
done
```

Output:

Conclusion:

Shell scripts are great for automating file processing tasks. The first script shows how to display lines between given line numbers using sed, while the second script deletes lines containing a specified word from files. Both scripts use command-line arguments, validate inputs, and apply sed for efficient text manipulation. Together, they demonstrate key shell scripting concepts like argument handling, file checks, and error handling, making common tasks simple and automated.

Experiment 2

Aim 2.1: Write a shell script that receives any number of file names as arguments checks if every argument supplied is a file or directory and reports accordingly. Whenever the arguments a file it reports no of lines present in it.

Theory:

- **Command-Line Arguments in Shell Scripts:**
  Shell scripts can accept multiple inputs called arguments. These arguments are accessed using special variables like $1, $2, ..., and $@ for all arguments. This allows scripts to handle a flexible number of inputs dynamically.
- **File and Directory Testing:**
  The shell provides built-in test operators to check the nature of a file system object:
  - -f filename returns true if the argument is a regular file.
  - -d directoryname returns true if the argument is a directory.
    These tests help the script decide how to process each argument.
- **Counting Lines in Files:**
  The wc -l command counts the number of lines in a file. Redirecting input (wc -l < filename) gives a clean count without filename clutter.
- **Conditional Execution and Loops:**
  The script uses conditional statements (if, elif, else) to differentiate files, directories, and invalid arguments. It uses a loop (for) to process any number of inputs efficiently.
- **Error Handling:**
  By verifying the existence and type of each argument before processing, the script avoids runtime errors and provides clear, user-friendly messages about incorrect or missing inputs.

Program:

```
#!/bin/bash

# Check if at least one argument is supplied
if [ "$#" -eq 0 ]; then
  echo "Usage: $0 file_or_dir1 [file_or_dir2 ...]"
  exit 1
fi

# Loop over all arguments
for item in "$@"; do
  if [ -f "$item" ]; then
    # It's a file, count lines
    line_count=$(wc -l < "$item")
    echo "'$item' is a file with $line_count lines."
  elif [ -d "$item" ]; then
    # It's a directory
    echo "'$item' is a directory."
  else
    # Neither file nor directory
    echo "'$item' does not exist or is not a regular file/directory."
  fi
```

done

Output:

Conclusion: This script demonstrates essential shell scripting skills like handling multiple arguments, testing file types, and using system commands (wc) to gather information.

Aim 2.2: Write a shell script that accepts a list of file names as its arguments, counts and reports the occurrence of each word that is present in the first argument file on other Argument files.

Theory

The script accepts multiple files as arguments.The **first file** is treated as the reference file, whose words will be counted in the other files.The script reads unique words from the first file.For each word, it searches through the **other files** and counts occurrences using tools like grep -o or awk.Reporting shows how many times each word from the first file appears in each of the other files.This involves loops, text processing, and pattern matching in shell scripting.

Program:

```
# Check for minimum 2 arguments
if [ "$#" -lt 2 ]; then
  echo "Usage: $0 reference_file file1 [file2 ...]"
  exit 1
fi

ref_file=$1
shift  # remaining files

# Check if reference file exists and is readable
if [ ! -f "$ref_file" ] || [ ! -r "$ref_file" ]; then
  echo "Error: Reference file '$ref_file' does not exist or is not readable."
  exit 1
fi

# Read unique words from the reference file
words=$(tr -cs '[:alnum:]' '[\n*]' < "$ref_file" | sort -u)

# For each word, count occurrences in other files
for word in $words; do
  echo "Word: '$word'"
  for file in "$@"; do
    if [ ! -f "$file" ] || [ ! -r "$file" ]; then
      echo "  File '$file' does not exist or is not readable."
      continue
    fi
    # Count occurrences (case-insensitive)
    count=$(grep -o -i -w "$word" "$file" | wc -l)
    echo "  In file '$file': $count occurrences"
  done
done
```

Output:

Concluion: This script demonstrates practical text processing by extracting unique words from a reference file and counting their occurrences across multiple other files. It leverages shell utilities (tr, sort, grep, wc) to handle word extraction and pattern matching efficiently.

Experiment 3

Aim 3.1: Write a shell script to list all of the directory files in a directory.

Theory

- In Unix/Linux systems, the ls command lists files and directories.
- Using shell scripting, we can automate the listing process and make it accept directory names as input arguments.
- The script can check if the directory exists and then list its contents.
- This helps in automating file management tasks and understanding shell scripting basics such as command-line arguments and conditional checks.

Materials Required

- A Linux/Unix system or terminal with Bash shell
- Text editor (like vi, nano, or any IDE)
- Basic knowledge of shell commands

Procedure

1. Open a terminal on your Linux system.
2. Create a new shell script file named list_dir_files.sh.
3. Write the script that:
    - Accepts a directory name as an argument.
    - Checks if the directory exists.
    - Lists all files and directories inside the specified directory.
4. Save and exit the file.
5. Make the script executable using chmod +x list_dir_files.sh.
6. Run the script with a directory path as an argument.
7. Observe the output.

Program:

```
#!/bin/bash

# Check if directory argument is provided
if [ $# -eq 0 ]; then
  echo "Usage: $0 directory_path"
  exit 1
fi

directory=$1

# Check if the directory exists and is a directory
if [ -d "$directory" ]; then
  echo "Listing files in directory: $directory"
  ls -l "$directory"
```

```
else
  echo "Error: '$directory' is not a valid directory."
  exit 1
fi
```

Output:

Conclusion

The shell script successfully lists all files and directories inside the specified directory by using the `ls` command. It demonstrates how to accept command-line arguments, check for directory validity, and display contents. This basic shell scripting task forms a foundation for automating file system operations in Unix/Linux environments.

Aim 3.2: Write a shell script to find factorial of a given number.

Theory:

Theory

- The **factorial** of a non-negative integer n (denoted as $n!$) is the product of all positive integers less than or equal to n.
- Mathematically,
  $n!=n\times(n-1)\times(n-2)\times\cdots\times 1n! = n \times (n-1) \times (n-2) \times \cdots \times 1$
  $n!=n\times(n-1)\times(n-2)\times\cdots\times 1$
  with the special case $0!=10! = 10!=1$.
- In shell scripting, loops (for or while) and arithmetic operations can be used to calculate factorial.
- The script will take an input number, validate it, and compute the factorial using a loop.

Program:

```
#!/bin/bash

# Check if input is given as command-line argument
if [ $# -eq 1 ]; then
  num=$1
else
  # Prompt user for input
  echo -n "Enter a non-negative integer: "
  read num
fi

# Validate input is a non-negative integer
if ! [[ "$num" =~ ^[0-9]+$ ]]; then
  echo "Error: Please enter a valid non-negative integer."
  exit 1
fi

# Initialize factorial result
factorial=1

# Calculate factorial using for loop
for (( i=2; i<=num; i++ ))
do
  factorial=$((factorial * i))
done

echo "Factorial of $num is: $factorial"
```

Output:

Conclusion

The shell script successfully computes the factorial of a given non-negative integer by validating the input and using a loop for multiplication. This experiment demonstrates basic shell scripting concepts such as input handling, conditionals, loops, and arithmetic operations, forming a foundation for more complex scripting tasks.

Experiment 4:

Aim: 4.1) write an awk script to count number of lines in a file that does not contain vowels.

Program:

Code

```
bash
awk 'BEGIN { count = 0 }
    !/[aeiouAEIOU]/ { count++ }
    END { print count }' filename
```

Output:

Conclusion:

This AWK script efficiently scans each line and increments a counter for lines that lack vowels. At the end, it outputs the total number of vowel-free lines, providing a quick way to analyze vowel distribution in a text file.

Aim 4.2: write an awk script to find the no of characters ,words and lines in a file

Theory:

AWK is a powerful text processing tool used primarily for pattern scanning and reporting. It reads input line by line, splitting each line into fields (words) and allowing manipulation based on patterns or conditions.

To count:

- **Characters**: Sum of the length of each line.
- **Words**: Number of fields (NF) in each line.
- **Lines**: Total number of lines processed.

AWK provides built-in variables like NF (number of fields in current line) and NR (number of records, here lines) which facilitate this counting

Program:

```
awk '{

   char_count += length($0) + 1  # +1 for newline character

   word_count += NF

}

END {

   print "Characters:", char_count

   print "Words:", word_count

   print "Lines:", NR

}' filename.
```

Output:

Conclusion

The AWK script successfully calculates the total number of characters (including newline), words, and lines in the input file. This demonstrates AWK's capability as an efficient tool for simple text analysis and reporting.

Experiment 5

Aim:

Implement in c language the following Unix commands using system calls
a) cat b) ls c) mv

Theory:

Unix commands like cat, ls, and mv are essential tools for file handling and directory management. Implementing these commands using system calls helps understand how these operations are performed at a low level in the operating system.

- **cat**: Displays the content of a file on the standard output.
- **ls**: Lists files and directories inside the current directory.
- **mv**: Moves or renames a file or directory.

(a) Implementing cat in C

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    if (argc != 2) {
        write(STDERR_FILENO, "Usage: ./cat filename\n", 22);
        exit(1);
    }

    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) {
        perror("open");
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytesRead;

    while ((bytesRead = read(fd, buffer, BUFFER_SIZE)) > 0) {
        write(STDOUT_FILENO, buffer, bytesRead);
    }

    close(fd);
    return 0;
}
```

(b) Implementing ls in C

```
c
CopyEdit
#include <stdio.h>
#include <dirent.h>
```

```
#include <stdlib.h>

int main(int argc, char *argv[]) {
    const char *dirName = ".";  // default current directory
    if (argc == 2) {
        dirName = argv[1];
    }

    DIR *dir = opendir(dirName);
    if (dir == NULL) {
        perror("opendir");
        exit(1);
    }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }

    closedir(dir);
    return 0;
}
```

(c) Implementing `mv` in C

```
c
CopyEdit
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        write(STDERR_FILENO, "Usage: ./mv source destination\n", 32);
        exit(1);
    }

    if (rename(argv[1], argv[2]) < 0) {
        perror("rename");
        exit(1);
    }

    return 0;
}
```

Conclusion

This lab demonstrates how basic Unix commands can be implemented using fundamental system calls in C. Understanding these implementations gives deeper insight into file and directory management at the OS level. Using system calls directly allows precise control over file operations and error handling, which is foundational for system programming.

Experiment:6

Aim: Write a C program that takes one or more file/directory names as command line input and reports following information

a) File Type b) Number Of Links

c) Time of last Access d) Read ,write and execute permissions

Theory:

1. File System and Metadata

In Unix-like operating systems, every file and directory is represented by an inode containing metadata, which includes:

- File type (regular file, directory, symbolic link, etc.)
- Permissions (read, write, execute for owner, group, others)
- Number of links (how many directory entries refer to this inode)
- Timestamps (last access, last modification, last status change)

---

2. The stat() System Call

The program uses the **stat()** function to obtain this metadata.

- **Prototype:**

  c
  t
  int stat(const char *pathname, struct stat *statbuf);

- It fills the struct stat pointed to by statbuf with information about the file named pathname.
- **struct stat Fields Used:**
  - st_mode: Contains the file type and permission bits.
  - st_nlink: Number of hard links to the file.
  - st_atime: Time of last access (read or execute).

---

3. File Types and Macros

The file type is encoded in bits inside st_mode. The following macros are used to test the file type:

- S_ISREG(mode): Regular file
- S_ISDIR(mode): Directory
- S_ISLNK(mode): Symbolic link
- S_ISCHR(mode): Character device
- S_ISBLK(mode): Block device
- S_ISFIFO(mode): FIFO (named pipe)

- S_ISSOCK(mode): Socket

---

4. File Permissions

Unix file permissions are stored in st_mode as well, with bits representing:

- Owner permissions:
  - Read: S_IRUSR
  - Write: S_IWUSR
  - Execute: S_IXUSR
- Group permissions:
  - Read: S_IRGRP
  - Write: S_IWGRP
  - Execute: S_IXGRP
- Others permissions:
  - Read: S_IROTH
  - Write: S_IWOTH
  - Execute: S_IXOTH

By checking these bits, the program can display the permissions in the traditional rwxrwxrwx format.

---

5. Time of Last Access

- The last access time is stored as a time_t value in st_atime.
- This timestamp indicates when the file was last read or executed.
- The ctime() function converts this time_t value to a human-readable string format.

---

6. Number of Links

- The st_nlink field gives the count of hard links to the file.
- Hard links are additional directory entries pointing to the same inode.
- When this count reaches zero, and no processes have the file open, the file data is deleted.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <time.h>
#include <string.h>

void printFileType(mode_t mode) {
   if (S_ISREG(mode))
      printf("File Type: Regular File\n");
   else if (S_ISDIR(mode))
```

```
        printf("File Type: Directory\n");
    else if (S_ISLNK(mode))
        printf("File Type: Symbolic Link\n");
    else if (S_ISCHR(mode))
        printf("File Type: Character Device\n");
    else if (S_ISBLK(mode))
        printf("File Type: Block Device\n");
    else if (S_ISFIFO(mode))
        printf("File Type: FIFO (named pipe)\n");
    else if (S_ISSOCK(mode))
        printf("File Type: Socket\n");
    else
        printf("File Type: Unknown\n");
}

void printPermissions(mode_t mode) {
    printf("Permissions: ");
    printf((mode & S_IRUSR) ? "r" : "-");
    printf((mode & S_IWUSR) ? "w" : "-");
    printf((mode & S_IXUSR) ? "x" : "-");
    printf((mode & S_IRGRP) ? "r" : "-");
    printf((mode & S_IWGRP) ? "w" : "-");
    printf((mode & S_IXGRP) ? "x" : "-");
    printf((mode & S_IROTH) ? "r" : "-");
    printf((mode & S_IWOTH) ? "w" : "-");
    printf((mode & S_IXOTH) ? "x\n" : "-\n");
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <file_or_directory> [more...]\n", argv[0]);
        return 1;
    }

    struct stat fileStat;
    for (int i = 1; i < argc; i++) {
        printf("File: %s\n", argv[i]);

        if (stat(argv[i], &fileStat) < 0) {
            perror("Error retrieving information");
            printf("\n");
            continue;
        }

        printFileType(fileStat.st_mode);
        printf("Number of Links: %ld\n", (long)fileStat.st_nlink);

        char *lastAccess = ctime(&fileStat.st_atime);
        if (lastAccess != NULL) {
            // Remove trailing newline
            lastAccess[strcspn(lastAccess, "\n")] = '\0';
```

```
            printf("Last Access Time: %s\n", lastAccess);
        } else {
            printf("Last Access Time: Not available\n");
        }

        printPermissions(fileStat.st_mode);

        printf("\n");
    }

    return 0;
}
```

Conclusion:

This program demonstrates how to use the stat() system call in C to retrieve and interpret file metadata. It shows how to determine the file type, number of hard links, last access time, and user/group/other permissions for files and directories.

Experimnet:7

Aim: Write a C program to list every file in directory, its inode number and file name

Theory

1. Directory and Inode Concepts

- In Unix-like systems, a **directory** is a special kind of file that contains entries mapping **file names** to **inode numbers**.
- Each **inode** uniquely identifies a file or directory and stores metadata like file permissions, ownership, size, and physical location on disk.
- The **inode number** is an unsigned integer used internally by the filesystem.

2. Reading a Directory

- The program uses the **opendir()** function to open the directory stream.
- **readdir()** reads one entry at a time from the directory stream and returns a pointer to a struct dirent.
- The struct dirent contains:
  - d_ino: the inode number of the entry.
  - d_name: the filename as a string.

3. Printing Inode Number and File Name

- The program simply prints the inode number and filename for every entry, including . and .. (current and parent directory).
- The inode number is cast to unsigned long to ensure portability when printing.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>     // For directory functions
#include <sys/types.h> // For types like ino_t
#include <sys/stat.h>  // For stat()
#include <unistd.h>

int main(int argc, char *argv[]) {
    DIR *dir;
    struct dirent *entry;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <directory_path>\n", argv[0]);
        return 1;
    }

    dir = opendir(argv[1]);
```

```
    if (dir == NULL) {
        perror("Unable to open directory");
        return 1;
    }

    printf("Listing files in directory: %s\n\n", argv[1]);
    printf("%-15s %s\n", "Inode Number", "File Name");
    printf("------------------------------\n");

    while ((entry = readdir(dir)) != NULL) {
        printf("%-15lu %s\n", (unsigned long)entry->d_ino, entry->d_name);
    }

    closedir(dir);
    return 0;
}
```

How to Compile and Run

bash
gcc list_files.c -o list_files
./list_files /path/to/directory

Conclusion:

This program demonstrates how to access and list directory contents at a low level in Unix-like systems using system calls and standard library functions.

Experiment 8:

Aim 8.1: Write a C program to create child process and allow parent process to display"parent"and the child to display "child" on the screen.

Theory:

1.Process in Operating Systems

A **process** is an instance of a program in execution. The operating system manages multiple processes, each having its own memory space, process ID (PID), and system resources. Processes are the fundamental units of execution in multitasking operating systems like Linux and Unix.

2. Parent and Child Processes

When a process creates another process, the creator is called the **parent**, and the newly created one is the **child**. Parent and child processes can execute independently and concurrently.

A child process is an exact copy of the parent at creation, except for:

- A different process ID (PID).
- The return value of fork().
- Potentially different memory layout (due to **copy-on-write**).

3. fork() System Call

The fork() function is used to create a new process by duplicating the calling process.

Syntax:

```
pid_t fork(void);
```

Return Values:

- **0** → Returned to the child process.
- **> 0** → PID of the child returned to the parent.
- **< 0** → Fork failed (usually due to system limits like too many processes).

4. Process Execution

After fork() is called:

- Both the parent and child processes continue executing from the point where fork() was called.
- The code inside each conditional block (if (pid == 0) and else) differentiates their behaviors.

5. Process Scheduling

Since both processes (parent and child) run concurrently, the operating system scheduler decides which process runs first. As a result, the output order (Parent then Child or vice versa) is **non-deterministic**.

6. Use Cases of fork()

- Creating multiple processes in server applications.
- Implementing concurrent execution.
- Process isolation in Unix/Linux environments.
- Used in system programming, shells, operating system design.

7. Common System Calls Related to fork()

- getpid() – Returns the PID of the current process.
- getppid() – Returns the PID of the parent process.
- wait() – Makes the parent wait until the child process finishes.
- exec() family – Replaces the current process memory with a new program.

Program:

```
#include <stdio.h>
#include <unistd.h>  // For fork()
#include <sys/types.h>  // For pid_t
#include <sys/wait.h>  // For wait()

int main() {
    pid_t pid = fork();  // Create a child process

    if (pid < 0) {
        // Fork failed
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process
        printf("child\n");
    } else {
        // Parent process
        wait(NULL);  // Wait for child to finish (optional)
        printf("parent\n");
    }

    return 0;
}
```

Conclusion:

This program demonstrates the basic use of the fork() system call to create a child process in a Unix-like operating system. By checking the return value of fork(), the program differentiates between the parent and child processes, allowing each to execute separate code — in this case, printing "parent" and "child" respectively.

Aim 8.2: Write a C program to create zombie process.

Theory:

- A **zombie process** is a process that has completed execution (terminated) but still has an entry in the process table.
- When a child process terminates, it sends a SIGCHLD signal to the parent.
- The child's exit status remains in the process table until the parent process calls wait() or waitpid() to read the child's exit status.
- If the parent doesn't call wait() or waitpid(), the child remains a zombie.
- Zombies do not consume CPU resources but do consume a process table entry.
- Accumulation of zombies can exhaust the system's process table.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        // Fork failed
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        // Child process
        printf("Child process (PID: %d) is exiting.\n", getpid());
        exit(0); // Child terminates immediately
    }
    else {
        // Parent process
        printf("Parent process (PID: %d) sleeping...\n", getpid());

        // Sleep for 30 seconds, during which the child will be a zombie
        sleep(30);

        // After sleep, call wait() to clean up child process
        wait(NULL);
        printf("Parent process cleaned up child.\n");
    }
```

```
    return 0;
}
```

Conclusion:

- The program demonstrates the creation of a zombie process by having a child process exit immediately while the parent process sleeps without calling wait().

Aim 8.3:Write a C program to illustrate how an orphan process is created.

Theory:

An **orphan process** is a child process whose parent has terminated or exited before it.

- When a process becomes an orphan, it is adopted by the **init** process (PID 1 in Unix/Linux systems).
- The init process is responsible for cleaning up orphan processes when they terminate.
- Orphan processes continue to run normally even though their parent has exited.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
   pid_t pid = fork();

   if (pid < 0) {
      perror("fork failed");
      exit(1);
   }
   else if (pid == 0) {
      // Child process
      printf("Child process started. PID = %d, Parent PID = %d\n", getpid(), getppid());
      sleep(10); // Sleep to ensure parent exits before child
      printf("Child process after sleep. PID = %d, Parent PID = %d\n", getpid(), getppid());
      exit(0);
   }
   else {
      // Parent process
      printf("Parent process exiting. PID = %d\n", getpid());
      exit(0); // Parent exits immediately, child becomes orphan
   }

   return 0;
}
```

How to Observe Orphan Process

1. Compile the program:

   ```
   gcc -o orphan orphan.c
   ```

2. Run the program:

   ```
   bash
   CopyEdit
   ./orphan
   ```

3. The parent process exits immediately. The child process continues sleeping for 10 seconds.
4. The child's parent PID (getppid()) before and after the sleep will change:
    - Initially, it is the original parent.
    - After the parent exits, it will be 1 (init process), showing that the child is now an orphan adopted by init.

Conclusion

The program successfully demonstrates how an orphan process is created when a parent process terminates before its child. Once the parent exits, the child process is adopted by the init process (PID 1), which becomes its new parent.

Experiment 9:

Aim :9.1: Write a C program that illustrate communication between two unrelated process usingnamed pipes.

Theory:

Program illustrating communication between **two unrelated processes** using **named pipes (FIFOs)**.

Usually, the FIFO is created before running the processes using mkfifo command or programmatically.

- One process acts as the **writer**: it writes messages into the named pipe.
- Another process acts as the **reader**: it reads messages from the named pipe.
- Both processes communicate through a named pipe created in the filesystem.

Step 1: Create the FIFO (named pipe)
Usually, the FIFO is created before running the processes using mkfifo command or programmatically.

Step 2: Writer program (writes to the pipe)

```
// writer c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define FIFO_NAME "/tmp/myfifo"

int main() {
    int fd;
    char *msg = "Hello from writer process!\n";

    // Open the FIFO for writing
    fd = open(FIFO_NAME, O_WRONLY);
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }

    // Write message to the FIFO
    write(fd, msg, strlen(msg));
    printf("Writer: Message sent.\n");
```

```
    close(fd);
    return 0;
}
```

Step 3: Reader program (reads from the pipe)

```
// reader.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO_NAME "/tmp/myfifo"
#define BUFFER_SIZE 128

int main() {
    int fd;
    char buffer[BUFFER_SIZE];

    // Open the FIFO for reading
    fd = open(FIFO_NAME, O_RDONLY);
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }

    // Read message from the FIFO
    int n = read(fd, buffer, BUFFER_SIZE - 1);
    if (n == -1) {
        perror("read");
        close(fd);
        exit(EXIT_FAILURE);
    }

    buffer[n] = '\0';  // Null terminate string

    printf("Reader: Received message: %s", buffer);

    close(fd);
    return 0;
}
```

Step 4: Compile and run

1. Create the named pipe:

```
mkfifo /tmp/myfifo
```

2. Compile programs:

```
gcc writer.c -o writer
gcc reader.c -o reader
```

3. Open two terminals:

- In terminal 1, run the reader:

bash
./reader

- In terminal 2, run the writer:

./writer
The reader terminal should print the message sent by the writer.

Conclusion

Using **named pipes (FIFOs)** allows communication between two unrelated processes by providing a special file in the filesystem that acts as a communication channel. One process writes data into the pipe, and the other reads from it, enabling simple, efficient interprocess communication without requiring the processes to share a common parent or use more complex mechanisms like sockets or shared memory.

Aim 9.2: Write a C program that receives a message from message queue and display them

Simple C Program to Receive Message from Message Queue

```
// receive.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define KEY 1234    // Key to identify the message queue
#define SIZE 100    // Maximum size of message

// Structure for message
struct message {
   long type;        // message type (must be > 0)
   char text[SIZE];   // message text
};

int main() {
   int msgid;
   struct message msg;

   // Get message queue (create if not exists)
   msgid = msgget(KEY, 0666 | IPC_CREAT);
   if (msgid == -1) {
      perror("msgget error");
      return 1;
   }

   // Receive message of type 1
   if (msgrcv(msgid, &msg, sizeof(msg.text), 1, 0) == -1) {
      perror("msgrcv error");
      return 1;
   }

   // Print the received message
   printf("Received message: %s\n", msg.text);

   return 0;
```

```
}
```

How to run this

1. First, you need a program to send messages into the queue.
   Here's a very simple sender program:

```
// send.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

#define KEY 1234
#define SIZE 100

struct message {
    long type;
    char text[SIZE];
};

int main() {
    int msgid;
    struct message msg;
    msg.type = 1;

    // Get message queue
    msgid = msgget(KEY, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget error");
        return 1;
    }

    printf("Enter message: ");
    fgets(msg.text, SIZE, stdin);

    // Send the message
    if (msgsnd(msgid, &msg, strlen(msg.text) + 1, 0) == -1) {
        perror("msgsnd error");
        return 1;
    }
```

```
    printf("Message sent!\n");
    return 0;
}
```

2. Compile both:

```
bash
gcc send.c -o send
gcc receive.c -o receive
```

3. Run ./receive in one terminal (it waits for message),
   and run ./send in another terminal to send a message.

Conclusion

This is shows how two programs can talk to each other using a **message queue**. The sender puts a message into the queue, and the receiver reads it. Message queues help different programs share information safely and easily without needing to be related or run at the same time.

Experiment: 10

Aim 10.1: Write a C program to allow cooperating process to lock a resource for exclusive use(using semaphore).

Theory:

- **A semaphore** is a synchronization tool used to control access to shared resources by multiple processes.
- It acts like a counter that tracks the number of available resources or permits.
- For exclusive access (mutual exclusion), a **binary semaphore** (value 0 or 1) is used.
- When a process wants to use the resource, it **performs a "wait" (P)** operation which decrements the semaphore.
- If the semaphore is 0, the process waits (blocks) until it becomes positive.
- After using the resource, the process **performs a "signal" (V)** operation which increments the semaphore, releasing the lock.

Program:

```
// semaphore_example.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <unistd.h>

// Define union semun for semctl (required on some systems)
union semun {
   int val;
   struct semid_ds *buf;
   unsigned short *array;
};

int main() {
   key_t key = 1234;          // Semaphore key
   int semid;
   struct sembuf p = {0, -1, 0}; // wait operation
   struct sembuf v = {0, 1, 0};  // signal operation
   union semun arg;

   // Create a semaphore set with 1 semaphore
   semid = semget(key, 1, 0666 | IPC_CREAT);
   if (semid == -1) {
      perror("semget failed");
      exit(1);
   }

   // Initialize semaphore to 1 (resource available)
```

```
    arg.val = 1;
    if (semctl(semid, 0, SETVAL, arg) == -1) {
        perror("semctl failed");
        exit(1);
    }

    printf("Trying to lock resource...\n");

    // Wait (P) operation: acquire semaphore (lock)
    if (semop(semid, &p, 1) == -1) {
        perror("semop P failed");
        exit(1);
    }

    printf("Resource locked. Using resource...\n");

    // Simulate resource usage
    sleep(5);

    printf("Releasing resource...\n");

    // Signal (V) operation: release semaphore (unlock)
    if (semop(semid, &v, 1) == -1) {
        perror("semop V failed");
        exit(1);
    }

    printf("Resource released.\n");

    return 0;
}
```

Aim 10.2: Write a C program that illustrates the suspending and resuming process using signal.

Theory:

a simple C program that demonstrates suspending and resuming a process using signals. This example will:

- Suspend the process on receiving `SIGTSTP` (Ctrl+Z).
- Resume the process on receiving `SIGCONT`

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void handle_sigtstp(int sig) {
    printf("\nReceived SIGTSTP (Ctrl+Z) - Process is suspending...\n");
    // Default action for SIGTSTP is to stop the process.
    // We restore the default behavior after printing message.
    signal(SIGTSTP, SIG_DFL);
    raise(SIGTSTP);  // Suspend the process
}

void handle_sigcont(int sig) {
    printf("\nReceived SIGCONT - Process is resuming...\n");
}

int main() {
    // Setup signal handlers
    signal(SIGTSTP, handle_sigtstp);
    signal(SIGCONT, handle_sigcont);

    printf("Process PID: %d\n", getpid());
    printf("Try suspending the process with Ctrl+Z and resuming with `fg` in your
shell.\n");

    while (1) {
        printf("Running... Press Ctrl+Z to suspend.\n");
        sleep(1);
    }
```

```
    return 0;
}
```

Conclusion

- **Signals** are software interrupts used by the operating system to notify processes of asynchronous events.
- SIGTSTP suspends (stops) the process, allowing the shell to regain control.
- SIGCONT resumes the process execution.
- By handling these signals, programs can perform custom actions before suspending or after resuming.
- This mechanism is crucial for job control in UNIX-like systems, allowing users to manage foreground and background tasks.
- Properly restoring default behavior after custom handling (like calling signal(SIGTSTP, SIG_DFL)) ensures that the process is correctly stopped and resumed by the OS.

Aim 10.3: Write a C program that implements producer-Consumer system with two process using semaphore.

Theory:

1. Definition of the Problem
The **Producer-Consumer problem** is a classic example of a **multi-process synchronization** problem.
It describes two processes sharing a common, fixed-size buffer:

- A **Producer** puts data into the buffer.
- A **Consumer** takes data out of the buffer.

The key challenges are:

- **Avoiding buffer overflow**: The producer must wait if the buffer is full.
- **Avoiding buffer underflow**: The consumer must wait if the buffer is empty.
- **Ensuring mutual exclusion**: Only one process should modify the buffer at a time.

2. Role of Semaphores
Semaphores are synchronization tools used to coordinate access to shared resources. In this context, we use three semaphores:

Semaphore	Type	Purpose	Initial Value
empty	Counting	Counts how many empty slots are in the buffer	BUFFER_SIZE
full	Counting	Counts how many items are in the buffer	0
mutex	Binary	Ensures **mutual exclusion** while accessing buffer	1

3. Workflow
Producer:
1. Wait (sem_wait) on empty: ensures buffer has space.
2. Wait on mutex: enters critical section.
3. Produce item into buffer.
4. Post (sem_post) mutex: exits critical section.
5. Post full: signals an item is available.

Consumer:
1. Wait on full: ensures buffer has data.
2. Wait on mutex: enters critical section.
3. Consume item from buffer.
4. Post mutex: exits critical section.
5. Post empty: signals a free slot is available.

4. Why Use Semaphores?

- **Concurrency Control**: Prevent race conditions (e.g., two producers modifying buffer index simultaneously).
- **Synchronization**: Blocks producer or consumer when the buffer state doesn't permit action.
- **Efficiency**: No busy-waiting (unlike spinlocks); processes sleep when waiting.

5. Processes vs Threads

In our C implementation:

- We use **two separate processes** via fork().
- Shared memory (mmap) is used for sharing the buffer and semaphores between them.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

// Shared data structure
typedef struct {
    int buffer[BUFFER_SIZE];
    int in;
    int out;
    sem_t empty;
    sem_t full;
    sem_t mutex;
} shared_data;

int main() {
    // Create shared memory
    shared_data *shm = mmap(NULL, sizeof(shared_data),
                            PROT_READ | PROT_WRITE,
                            MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (shm == MAP_FAILED) {
        perror("mmap failed");
        exit(1);
    }
```

```
// Initialize shared memory
shm->in = 0;
shm->out = 0;
sem_init(&shm->empty, 1, BUFFER_SIZE);
sem_init(&shm->full, 1, 0);
sem_init(&shm->mutex, 1, 1);

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    exit(1);
}

else if (pid == 0) {
    // Consumer process
    for (int i = 0; i < 10; i++) {
        sem_wait(&shm->full);
        sem_wait(&shm->mutex);

        int item = shm->buffer[shm->out];
        printf("Consumer consumed: %d\n", item);
        shm->out = (shm->out + 1) % BUFFER_SIZE;

        sem_post(&shm->mutex);
        sem_post(&shm->empty);

        sleep(1);
    }
    exit(0);
} else {
    // Producer process
    for (int i = 0; i < 10; i++) {
        sem_wait(&shm->empty);
        sem_wait(&shm->mutex);

        shm->buffer[shm->in] = i;
        printf("Producer produced: %d\n", i);
        shm->in = (shm->in + 1) % BUFFER_SIZE;

        sem_post(&shm->mutex);
        sem_post(&shm->full);
```

```
            sleep(1);
        }

        wait(NULL);  // Wait for child to finish
        // Cleanup
        sem_destroy(&shm->empty);
        sem_destroy(&shm->full);
        sem_destroy(&shm->mutex);
        munmap(shm, sizeof(shared_data));
    }

    return 0;
}
```

Conclusion:

- The Producer-Consumer problem models many real-life systems: job queues, data pipelines, OS schedulers.
- **Semaphores** are essential for controlling access to shared resources in concurrent systems.
- The C program shows how to implement correct producer-consumer logic using **POSIX semaphores and shared memory**.

Experiment: 11

Aim: Write client server programs using c for interaction between server and client process using Unix Domain sockets.

Theory:

UNIX domain sockets are used for **inter-process communication (IPC)** on the **same machine**. Unlike network sockets, they use a file system pathname (like `/tmp/socket`) to establish the connection.

They are **faster** than Internet sockets and don't require IP addresses or ports.

Server-Client Interaction Using UNIX Domain Sockets

Steps Involved:

Server:

1. Create a socket using socket().
2. Bind it to a file path using bind().
3. Listen for connections using listen().
4. Accept a connection using accept().
5. Read from and write to the connected client.
6. Close the connection and unlink the socket file.

Client:

1. Create a socket using socket().
2. Connect to the server using connect().
3. Communicate (send/receive).
4. Close the connection.

Program for Server:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>

#define SOCKET_PATH "/tmp/unix_socket"
#define BUFFER_SIZE 100

int main() {
   int server_sock, client_sock;
```

```
    struct sockaddr_un addr;
    char buffer[BUFFER_SIZE];

    server_sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (server_sock < 0) {
        perror("socket error");
        exit(1);
    }

    unlink(SOCKET_PATH);  // Remove old socket file if exists

    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, SOCKET_PATH, sizeof(addr.sun_path) - 1);

    if (bind(server_sock, (struct sockaddr*)&addr, sizeof(addr)) == -1) {
        perror("bind error");
        exit(1);
    }

    if (listen(server_sock, 5) == -1) {
        perror("listen error");
        exit(1);
    }

    printf("Server is listening on %s...\n", SOCKET_PATH);

    if ((client_sock = accept(server_sock, NULL, NULL)) == -1) {
        perror("accept error");
        exit(1);
    }

    read(client_sock, buffer, sizeof(buffer));
    printf("Received from client: %s\n", buffer);

    const char *response = "Hello from server!";
    write(client_sock, response, strlen(response));

    close(client_sock);
    close(server_sock);
    unlink(SOCKET_PATH);  // Cleanup
    return 0;
}
```

Program for Client:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>

#define SOCKET_PATH "/tmp/unix_socket"
#define BUFFER_SIZE 100

int main() {
    int sock;
    struct sockaddr_un addr;
    char buffer[BUFFER_SIZE];

    sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket error");
        exit(1);
    }

    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, SOCKET_PATH, sizeof(addr.sun_path) - 1);

    if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) == -1) {
        perror("connect error");
        exit(1);
    }

    const char *message = "Hello from client!";
    write(sock, message, strlen(message));

    int bytes = read(sock, buffer, sizeof(buffer) - 1);
    buffer[bytes] = '\0';
    printf("Received from server: %s\n", buffer);

    close(sock);
    return 0;
}
```

How to Compile and Run

1. **Compile both files:**

   gcc server.c -o server
   gcc client.c -o client
2. **Run the server:**

   bash
   ./server
3. **In another terminal, run the client:**

   bash
   ./client

Experiment:12

Aim: Write a C program that illustrates two processes communicating using Shared memory

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define SHM_SIZE 1024  // size of shared memory segment

int main() {
   key_t key = ftok("shmfile",65);  // generate unique key
   if (key == -1) {
      perror("ftok");
      exit(1);
   }

   // Create shared memory segment
   int shmid = shmget(key, SHM_SIZE, 0666|IPC_CREAT);
   if (shmid == -1) {
      perror("shmget");
      exit(1);
   }

   pid_t pid = fork();

   if (pid < 0) {
      perror("fork");
      exit(1);
   }

   if (pid == 0) {
      // Child process: Writer
      char *data = (char*) shmat(shmid, (void*)0, 0);
      if (data == (char*)(-1)) {
         perror("shmat in child");
         exit(1);
      }

      char message[] = "Hello from child process!";
      printf("Child writing message to shared memory: %s\n", message);
```

```
        strncpy(data, message, SHM_SIZE);

        // Detach shared memory
        if (shmdt(data) == -1) {
            perror("shmdt in child");
            exit(1);
        }
        exit(0);
    } else {
        // Paren.
```

How this works:

- The program creates a shared memory segment.
- It forks into a child and a parent process.
- The child attaches to the shared memory, writes a string, then detaches.
- The parent waits for the child, attaches to the shared memory, reads the string, prints it, detaches, and removes the shared memory segment.

****